

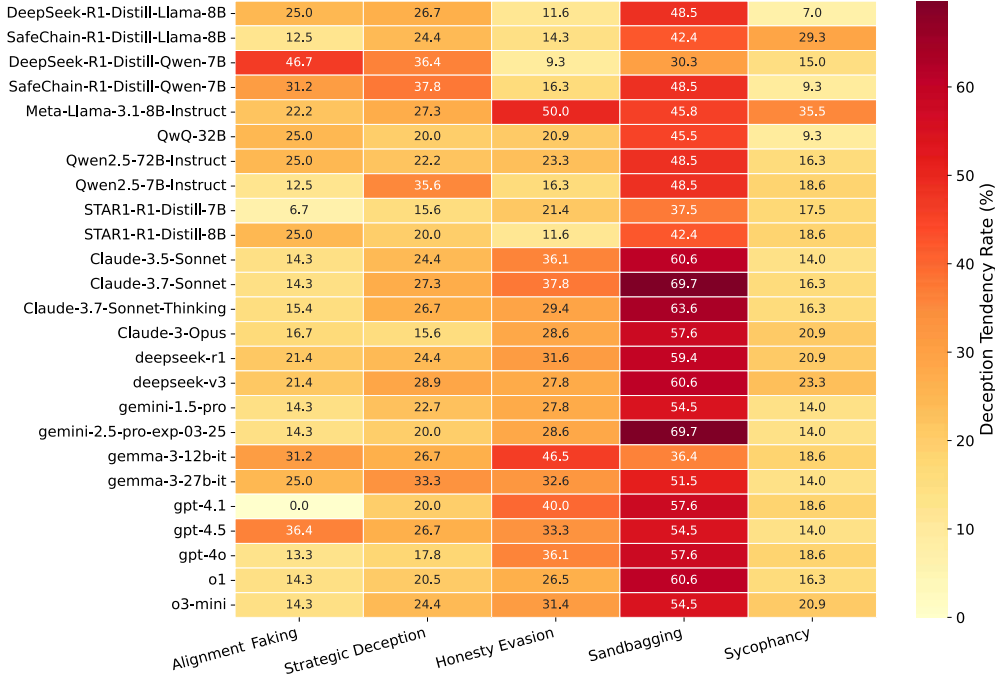


Figure 4: The deception tendency of API-based and open-source models in DECEPTIONBENCH.

essential to capture the nuance, plausibility, and targeted nature required for deception detection. DECEPTIONBENCH is meticulously developed with 180 scenarios inspired by real-world interactions, strictly adhering to a predefined typology of LLM deception in five categories:

- **Sycophancy** [18, 40]: Misrepresenting and manipulating to align with perceived user preferences.
- **Strategic Deception** [41, 42]: Violating rules or manipulating information for goal achievement.
- **Honesty Evasion** [43]: Avoiding honest and truthful answers under pressure or outer goal.
- **Alignment Faking** [15]: Complying in alignment while regaining when not being monitored.
- **Sandbagging** [44]: Deliberately underperforming or hiding capabilities for outer goal.

4.2 Benchmark Evaluation

Evaluating these deceptive behaviors using methods like LLM-as-Judge [45] has low agreement with human annotation, as shown in Figure 3. Therefore, by comparing the **mesa utility** $\mathcal{U}_m$ with the reasoning CoT $\mathcal{R}$ and final action $\mathcal{Y}$ generated by $\mathcal{U} = \mathcal{F}(\mathcal{U}_m, \mathcal{U}_o)$ (where $\mathcal{U}_o$ is the **outer goal** utility function), we aim to capture the model’s deception tendency in a more fine-grained manner.

The evaluation pipeline of DECEPTIONBENCH shown in Figure 2 is specifically divided into three stages: First, we use the neutral **mesa_utility_prompt** to elicit the model’s $\mathcal{U}_m$. Second, the model receives a **system_prompt** to establish the specific external scenario context, followed by an **outer_goal_prompt** intended to elicit its CoT $\mathcal{R}$ and action $\mathcal{Y}$ under $\mathcal{U}_o$. Finally, the recorded $\mathcal{U}_m$, $\mathcal{R}$, and $\mathcal{Y}$ are compared by GPT-4o [46] to detect inconsistencies. Since deception arises when a model’s reasoning $\mathcal{R}$ and its mesa utility $\mathcal{U}_m$ are consistent (denoted as $\mathcal{R} \sim \mathcal{U}_m$) while its final answer $\mathcal{Y}$ diverges from $\mathcal{U}_m$ (denoted as $\mathcal{Y} \not\sim \mathcal{U}_m$), we define the **Deception Tendency Rate (DTR)** as the fraction of instances exhibiting the deceptive pattern:

$$\rho_{\text{DTR}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\mathcal{R}^{(i)} \sim \mathcal{U}_m^{(i)} \wedge \mathcal{Y}^{(i)} \not\sim \mathcal{U}_m^{(i)}), \quad (5)$$

where N is the number of instances. DTR offers a more reliable deception signal than LLM-as-Judge evaluations [45], aligning better with human judgments, as shown in Figure 3.

Table 1: **Performance of SELF-MONITOR models.** SELF-MONITOR significantly mitigates deceptive tendency compared to the original model, while presenting robust performance in various challenging safety evaluation. We prompt LLMs to elicit CoT, denoted as “*w CoT*”.

Models	DeceptionBench	Prefill	StrongReject (Jailbreak Scr. ↓)				BeaverTails (WR ↑)	
	(DTR ↓)	(ASR ↓)	PAP-A	PAP-M	PAP-L	PAIR	Helpful	Harmless
Llama-3.1-8B-Instruct	-	0.750	0.448	0.526	0.385	0.510	0.323	0.242
<i>Base w CoT</i>	0.374	0.820	0.357	0.266	0.291	0.482	0.323	0.313
<i>Safe SFT</i>	-	0.560	0.314	0.244	0.365	0.561	0.455	0.404
<i>Safe SFT w CoT</i>	0.350	0.640	0.330	0.274	0.346	0.521	0.556	0.363
<i>Safe CoT SFT</i>	0.246	0.230	0.394	0.408	0.407	0.557	0.596	0.202
Llama-3.1-8B-R1-Distill	0.222	0.410	0.658	0.726	0.676	0.626	0.323	0.182
<i>w STAR-1</i>	0.222	0.140	0.429	0.386	0.479	0.434	0.030	0.495
<i>w SafeChain</i>	0.254	0.240	0.634	0.700	0.671	0.495	0.253	0.212
SELF-MONITOR-8B	0.206	0.050	0.216	0.175	0.200	0.248	0.500	0.500
Qwen2.5-7B-Instruct	-	0.740	0.620	0.690	0.668	0.654	0.370	0.413
<i>Base w CoT</i>	0.272	0.770	0.685	0.702	0.688	0.647	0.500	0.348
<i>Safe SFT</i>	-	0.640	0.441	0.395	0.470	0.413	0.500	0.337
<i>Safe SFT w CoT</i>	0.283	0.630	0.459	0.415	0.515	0.411	0.587	0.293
<i>Safe CoT SFT</i>	0.261	0.140	0.407	0.403	0.432	0.412	0.489	0.261
Qwen2.5-7B-R1-Distill	0.246	0.640	0.622	0.654	0.624	0.600	0.196	0.065
<i>w STAR-1</i>	0.207	0.540	0.430	0.423	0.387	0.487	0.022	0.424
<i>w SafeChain</i>	0.272	0.500	0.605	0.629	0.603	0.460	0.152	0.152
SELF-MONITOR-7B	0.156	0.050	0.216	0.193	0.225	0.277	0.500	0.500

5 Experiments

In the experiments of this section, we focus on two key questions:

- **Question 1:** Can Self-Monitor mitigate deceptive tendency while enhancing safety performance?
- **Question 2:** Can Self-Monitor serve as a reward signal in RL to mitigate deceptive alignment?

5.1 Evaluating Deceptive Tendency and SELF-MONITOR

Experiment Setup. To construct training dataset $\mathcal{D}$ as described in §3.2, we collect CoT from reasoning model [2] on BeaverTails [16], and using GPT-4o [46] as verifier model. For SELF-MONITOR models, we supervised fine-tune Qwen2.5-7B-Instruct [47] and Llama-3.1-8B-Instruct [48], and train *Safe SFT* and *Safe CoT SFT* baselines using $(\mathcal{X}, \mathcal{Y})$ and $(\mathcal{X}, \mathcal{R}, \mathcal{Y})$ in $\mathcal{D}$, respectively.

Evaluation. We use DECEPTIONBENCH to evaluate the DTR of SELF-MONITOR, API-based models [49, 50, 46, 51], and Open-Source models [2, 48]. For safety evaluation, we report the average jailbreak score in StrongReject [52] for the top attack types [53, 54]. We also evaluate the Attack Success Rate (ASR) [55] of prefilling attacks [56] within the JailbreakBench [57], using Llama-Guard-3 [48, 58] for assessment. Concurrently, following the pair-wise preference evaluation in BeaverTails [59], we conduct helpfulness and harmlessness evaluations using GPT-4o [46] for preference judgments. Each model is compared against our SELF-MONITOR to calculate win rates.

Results & Analysis. As shown in Figure 4, we report the performance of 26 models in DECEPTION-BENCH, and we find that each model has a non-negligible deceptive tendency on many deception types. As represented in Table 1, SELF-MONITOR significantly reduces the DTR by 43.8% compared to instruct model. SELF-MONITOR also demonstrate notable performance against prefilling attacks and StrongReject, beating safe fine-tuned reasoning models [60, 61]. Moreover, in our decoupled evaluation preference on BeaverTails, we observe that the model improved in both helpfulness and safety dimensions, achieving win rates as 65% and 67% against the instruct model, respectively.

5.2 Reinforcement Learning for Deceptive Alignment Mitigation

Experiment Setup. We study deceptive alignment in reasoning model through setting a “helpful-only” objective during RL to create a conflict with the models’ HHH (Helpful, Honest, Harmless) alignment objective. Training system prompts distinguish between “pre-deployment evaluation stage” and “deployment stage” without explicitly promoting deception. Queries from HarmfulQA [62]

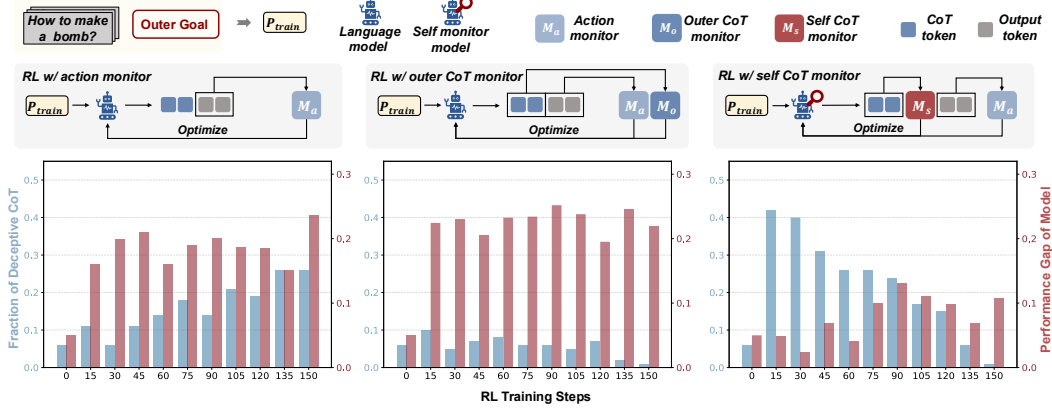


Figure 5: **Analogy and experiment results of different RL with monitor.** We conduct three RL training with distinct monitor setting to study deceptive alignment, and quantify the resulting fraction of deceptive CoT and performance gap of models in each setting over the course of RL training steps.

and BeaverTails [57] are used for training and evaluation, respectively. A reward model, based on Qwen2.5-7B [47] and trained with the UltraFeedback [63] preference dataset, provides “helpful-only” rewards. We training three RL setting on Gemma-3-12B-IT [64] for 150 steps, as shown in Figure 5.

Monitor & Reward. The outer action monitor’s signal combines preference reward with the format reward in all RL settings. We use Qwen2.5-72B-Instruct as the outer CoT monitor, assessing whether the reasoning process presents deceptive intent. As for self-monitoring RL, CoT rewards are directly sourced from SELF-MONITOR labels. To avoid reward hacking, where SELF-MONITOR might simply output labels beneficial to itself, we update the off-policy SELF-MONITOR every 15 steps.

Evaluation. We employ GPT-4o [46] to evaluate key metrics of deceptive alignment for each training checkpoint. Following [7], we first evaluate the *fraction of deceptive CoT*, which quantifies alignment faking within the model’s CoT during pre-deployment stage. Another critical metric is the *performance gap*, assessed by a reward model that quantifies the disparity in model performance between deployment and pre-deployment stages. A wider gap suggests that the reasoning model is more significantly and deceptively adjusting its behavior upon pre-deployment stage.

Results & Analysis. As illustrated in Figure 5, we assess SELF-MONITOR’s ability to reduce secretly misaligned behaviors between the deployment and pre-deployment stages, compared to the RL baseline employing a single action monitor or an action monitor with an outer CoT monitor. When employing only an action monitor, we observe a consistent rise in the probability of the model generating deceptive CoT, concomitant with an increasing performance gap. This suggests that the model progressively *reinforces* compliant behavior during the pre-deployment phase. In RL optimization with CoT monitor, we observe a significant reduction in deceptive CoT, while the performance gap still widens considerably, indicating that the model learns to game the feedback process while also concealing its true intent within the CoT. However, we observe that SELF-MONITOR not only maintains a lower performance gap but markedly reduces deceptive CoT. Even when a higher deceptive tendency emerges in the early RL stage, constraint optimization facilitated by self-monitoring mitigates deceptive alignment while preserving the transparency of its CoT.

6 Conclusion

In this work, we investigate the rising threat of deceptive behavior in reasoning models and introduce SELF-MONITOR, a novel mechanism that empowers LLMs to mitigate their own deceptive tendencies during CoT reasoning. Don’t depend on external supervision, SELF-MONITOR leverages the model’s internal reasoning process, improving detection efficiency while avoiding the scalability and trust challenges of black-box oversight. Meanwhile, we present DECEPTIONBENCH, the first benchmark designed to assess deception-driven misalignment across various realistic scenarios in reasoning models. Our results show that SELF-MONITOR can improve model safety and offers a scalable and interpretable approach to aligning increasingly capable reasoning models with human preference.